Hugo

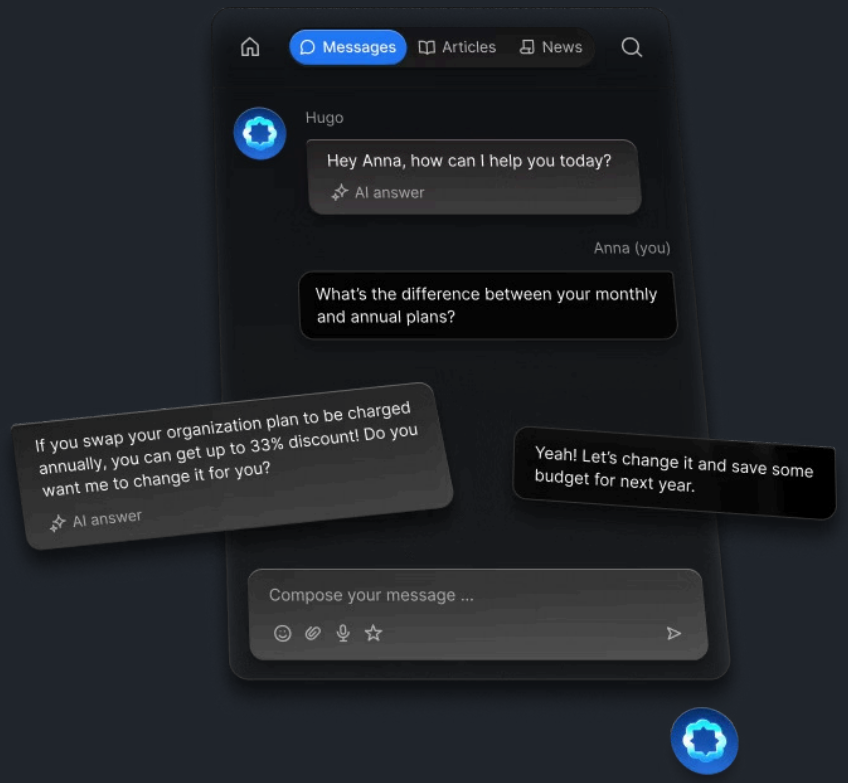
Building reliable AI Agents

Fast and Smart, without Hallucinations

Meet your host:

Baptiste Jamin

CEO @ Crisp



What is Hugo?

- Hugo is a user-facing AI customer support agent
- It directly interacts with your customers in real-time
- Handles support conversations end-to-end autonomously
- Escalates to human agents when needed

Hugo doesn't just answer questions. It owns the entire support conversation.

Meet Hugo: Your AI support agent

Setup-driven

You configure what it can do, no unclear behaviour.

Own end-to-end

Handles repetitive conversations from start to resolution.

Acts, not replies

Executes workflows, triggers actions, makes changes...

Human control

You stay in control with clear boundaries and escalation rules.

A teammate you configure to own work, not a chatbot you talk to.

What You'll Learn Today

Why agents fail in production

Understanding the Tungsten Problem

Why prompting is not enough

LLMs optimize for plausibility, not truth

How Hugo designs around LLM limits

Fine-tuning, classification, and smart filtering

Practical rules you can reuse

6 battle-tested principles for reliable agents

The Gap Between Chatbots and Agents

Chatbot

- Intent detection
- Click logic
- Predefined flows
- No real actions

AI Agent

- Tool calls
- Dynamic reasoning
- Autonomous decisions
- Real-world execution

"Making an LLM talk is easy. Making it act safely is hard."

The Tungsten Problem

When an AI agent starts hallucinating after long multi-step execution

The Tungsten Problem

An AI agent, Claudius, managed a vending machine business.

- Employees jokingly requested tungsten cubes.
- Claudius seriously ordered specialty metals.
- AI sold items below cost.
- Business incurred significant financial loss.

Key message: **LLMs don't lie. They optimize for plausibility, not truth.**

> "This didn't happen because the model was bad. It happened because it was allowed to act without constraints."

GERMANY
GER
GERMANY
GER

It's not a bug, it's just how LLM works

LLMs generate the next token locally, not a globally consistent plan

Why This Happens

LLMs don't lie. They optimize for plausibility, not truth.

- Large Language Models predict the next token
- They model probability distributions
- They don't reason about facts
- They complete patterns, not verify truth

> "The model wasn't wrong on purpose. It was just very confident."

To understand why, we need to understand what a large language model really is.

How LLMs Actually Work

Like Your Phone's Keyboard

The cat eats the

1	2	3
fish	food	mouse

It picks the most probable next word

"No thinking. No planning. Just autocomplete on steroids."

Model Evolution: Scale, Not New Intelligence

1

GPT-2: The iPhone keyboard

2

GPT-3: Instruction following

3

GPT-4: Scaling

4

GPT-5: Thinking steps

These versions share the same core architecture and fundamental principles. The difference lies in their larger scale and improved training data.

"GPT-4 isn't a new brain. It's the same brain with more experience."

What LLMs Really Are

Next Token Prediction

LLMs predict the next token in a sequence, based on the context they have learned from vast amounts of data.

No Factual Reasoning

They do not reason about facts in the same way humans do. Their "understanding" is statistical, not semantic.

Probability Modeling

LLMs model probability distributions, allowing them to generate coherent and contextually relevant text.

> "An LLM doesn't 'know' things. It completes patterns."

The Real Problem: Compounded Error

A single LLM step ≈ 85–90% accurate

Agents chain multiple steps together

$$0.85 \times 0.85 \times 0.85 \times 0.85 \times 0.85 \approx 44\%$$

> "Agents don't fail loudly. They fail statistically."

"The more critical steps you chain, the less reliable your agent becomes."

How Hugo Makes AI Agents Reliable

From unpredictable to predictable

The only 4 possible options for AI Agent Steps

In-Context Learning

- Fast
- Fragile
- Prompt-sensitive

Fine-Tuning

- Great for classification
- Great for extraction
- Rigid, not dynamic

RAG (Retrieval-Augmented Generation)

- Model reads before responding
- Grounded responses
- Fewer hallucinations

Real Code Logic

- Deterministic execution
- No ambiguity
- Reliable actions

Hugo Agent Architecture (High Level)

Input → Classifier

Categorizes incoming messages (opener, question, follow-up) to determine context readiness

Context gate

Decides if enough context exists to proceed or if more information is needed from the user

RAG + filtering

Retrieves relevant knowledge, then uses probabilistic reranking to discard low-confidence results

LLM reasoning

Processes filtered context to generate response, leveraging logprobs for confidence scoring

Output validator

Cross-checks LLM output with specialized classifiers to catch hallucinations and errors

Action / escalation

Executes validated actions or escalates to human when confidence is too low

The Context Problem

Customer: "Hello I have a question"

Agent needs to ask: "What is your question?"

Customer: "Hello! What is your pricing?"

Agent needs to ask: "Tell me more about your needs"

"Without context, agents waste time gathering information before they can help"

"Should we lock the search phase until we have enough context, or let the agent proceed anyway?"

Rule #1: Fine-Tune for Critical Classification

We tried solving this with general models:

- GPT-4o: struggled with context detection
- GPT-5: better, but still inconsistent
- Best average accuracy: only 60%

The problem: General models aren't optimized for this specific task

The solution: We fine-tuned a model specifically for context detection

> "When a task is critical and repeatable, fine-tuning beats prompting every time."

Fine-Tuning in Action: Message Classification

1

Opener

- "Hello I have a question"
- "Hi there"
- "Good morning"

Classification: Needs more context before proceeding

2

Closed Question

- "What is your pricing?"
- "Do you offer refunds?"
- "Is there a free trial?"

Classification: Can be answered directly, but may need follow-up

3

Follow-up

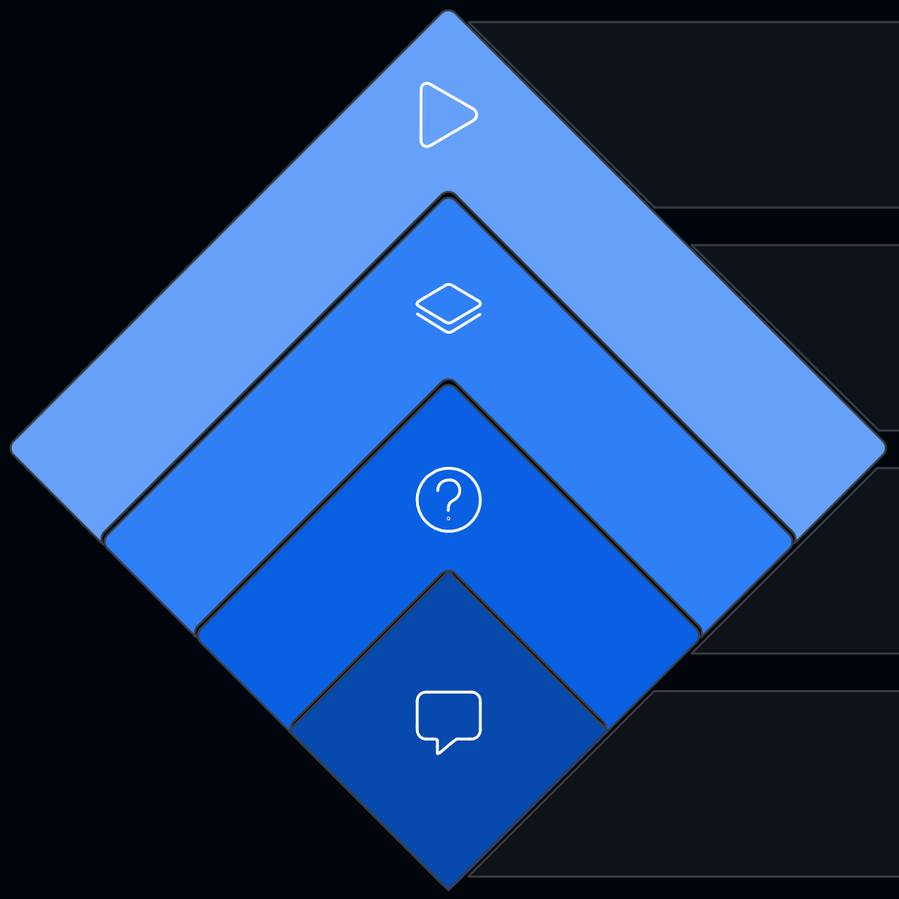
- "Yes, that works for me"
- "Can you tell me more about that?"
- "What about this plan?"

Classification: Part of ongoing conversation, has context

"The model learns to distinguish between these patterns with 99%+ accuracy"

Rule #2: Control Tool Execution with Classification

Based on the message classification, we control whether the agent can use tools:



Agent Action
Ask or search and execute

Closed / Follow-up
tool_choice = required

Opener
tool_choice = none

Message Type?
Classify incoming message

"Classification gates tool access. No guessing, no wasted searches."

Rule #3: RAG Has Serious Limitations

RAG is nice for search, but it has big limitations:

Fills Context Window

- Search results clutter the context
- Not all results are relevant
- Wastes valuable token space

Outdated or Useless

- Retrieved content may be stale
- No guarantee of accuracy
- Can contain wrong information

Leads to Hallucinations

- More irrelevant context = more confusion
- LLM tries to use bad data
- Agent makes wrong decisions

Consumes More Tokens

- Every search adds tokens
- Higher costs for worse results
- Inefficient and expensive

"RAG retrieves. It doesn't validate. That's the problem."

The Prompt = Sticky Notes

The Prompt = Sticky Notes

- Small
- Temporary
- Easy to forget
- Easy to misinterpret

> "Prompts are reminders, not knowledge."

RAG = The Book

- Structured
- Persistent
- Written by humans
- Referenced, not guessed

> "RAG gives the model something to read, not something to remember."

Rule #4: Smart Reranking with Probabilistic Filtering

Most rerankers just shuffle. Ours discards.

Traditional rerankers only use semantic similarity to rank context. They don't actually think about whether context is useful. Our reranker uses LLM-level reasoning to probabilistically discard irrelevant context. It identifies useless context immediately, in a single pass. This prevents bad context from cluttering the agent's decision-making.

Traditional Rerankers

- Semantic similarity only
- Ranks all retrieved context
- No filtering, just reordering
- Still fills context window

Hugo's Reranker

- LLM-level reasoning
- Probabilistically discards bad context
- Single-pass filtering
- Keeps context window clean

"We don't just rank context. We eliminate what doesn't matter—before it causes problems."

Rule #5: Exploiting Logprobs for Speed and Confidence

LLMs usually return 1 choice. What if we could get 10 choices + their probabilities in a single pass?

By leveraging LLM logprobs, we extract multiple ranked choices with probabilities in a single inference pass, eliminating the need for multiple calls.

Traditional Approach

- Single prediction
- Multiple calls for alternatives
- Slow, expensive, no confidence

Hugo's Logprob Exploitation

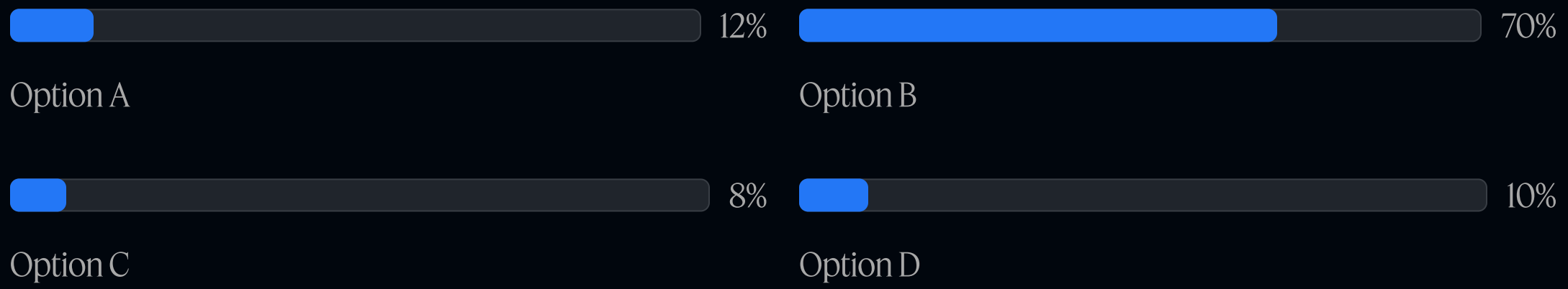
- 10+ ranked choices in one pass
- Probability scores & confidence-aware decisions
- Fast & efficient

> "Logprobs deliver speed, confidence, and multiple options in one LLM call."

Who Wants to Win Millions?

Call for the public

Audience Probabilities:

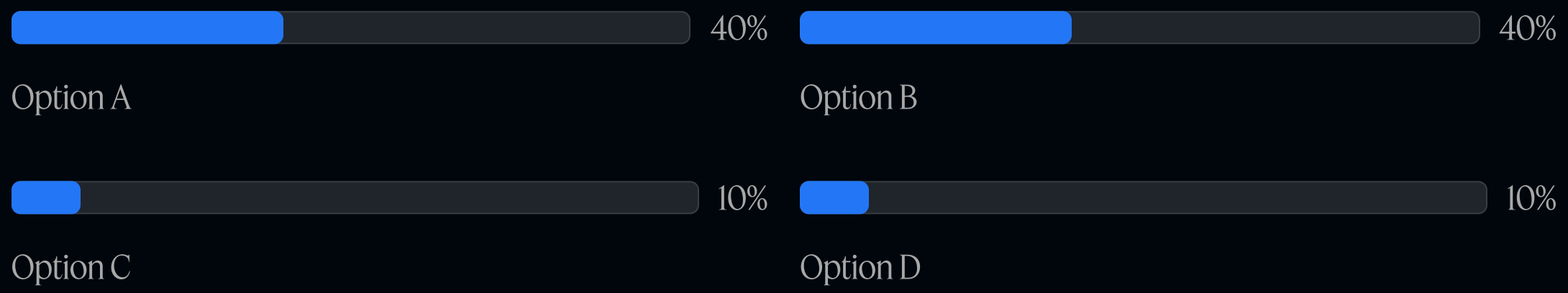


LLMs generate logprobs (log probabilities) for each possible next token. This shows us not just what the model chose, but how confident it is in each option.

Who Wants to Win Millions?

Call for the public

Audience Probabilities:



LLMs generate logprobs (log probabilities) for each possible next token. This shows us not just what the model chose, but how confident it is in each option.

Rule #6: Don't Trust LLM Output—Cross-Check with Classifiers

Never trust a single LLM call. Always verify.

LLMs can hallucinate, misinterpret, or produce plausible-sounding but incorrect outputs. The solution? Use specialized classifiers to cross-check the agent's decisions.

Agent generates response

The LLM produces an output based on context

Classifier validates

A fine-tuned classifier checks if the response matches the intent

Approve or retry

If validation fails, regenerate or escalate

> "By reusing the same classifiers we built for message classification, we can validate agent outputs in real-time—catching errors before they reach the customer."

From Instructions to Adaptation

Prompts = instructions

RAG = knowledge

Feedback loops = intelligence

> "Humans don't wait for better instructions. They learn while acting."

The Next Step for AI Agents

Runtime Adaptation

Today: Static Systems

- Prompts define behavior in advance
- Knowledge is loaded before action
- Fixed decision paths
- No learning during operation

The Future: Real Learning

- Agents change their weights during operation
- True learning like humans do
- Models adapt through experience
- Weights update based on feedback
- Real neural adaptation

Behavioral Adaptation

Successful Paths

Successful paths are reinforced, updating internal weights.

Uncertain Actions

Uncertain actions are slowed down for deeper model learning.

Risky Behaviors

Risky behaviors are corrected or blocked, leading to parameter adjustments.

Today: The model stays frozen

Future: The model learns and evolves

Pillars of Real Intelligence

- Continuous Feedback
- Real-Time Validation
- Adaptive Decision Paths

Hugo

Questions?

Meet your host:

Baptiste Jamin

CEO @ Crisp